

**Study of computational
efficiency in
Nonparametric
Inference modeling
using Distributed
Computing**

**Aurora Grefsrud, PhD
theory lecture, PCS951**

Uncertainties in data-driven infinite population inference

Presentation abstract:

In this project I look at different approaches to **nonparametric methods** of approximating **multivariate distributions** in "infinite population" high energy physics experiments, as well as how to calculate the **uncertainties** due to modeling and sampling effects. Because HEP experiments are extremely complex, the study is performed on a toy experiment, featuring a multivariate distribution of sufficient complexity to be non-trivial, but low enough dimensionality to be easily analyzed. The study is still in progress, but the aim is to **evaluate and compare** different neural network based machine learning methods as well as statistical methods using nonparametric Bayesian inference, to a baseline truth to check for convergence as well as possible bias. The goal of the study is to present clearly the **advantages and limitations** of different numerical methods for inference on "**noisy**" **distributions**, and evaluate which approach might be more useful for important HEP-specific tasks such as jet-tagging.

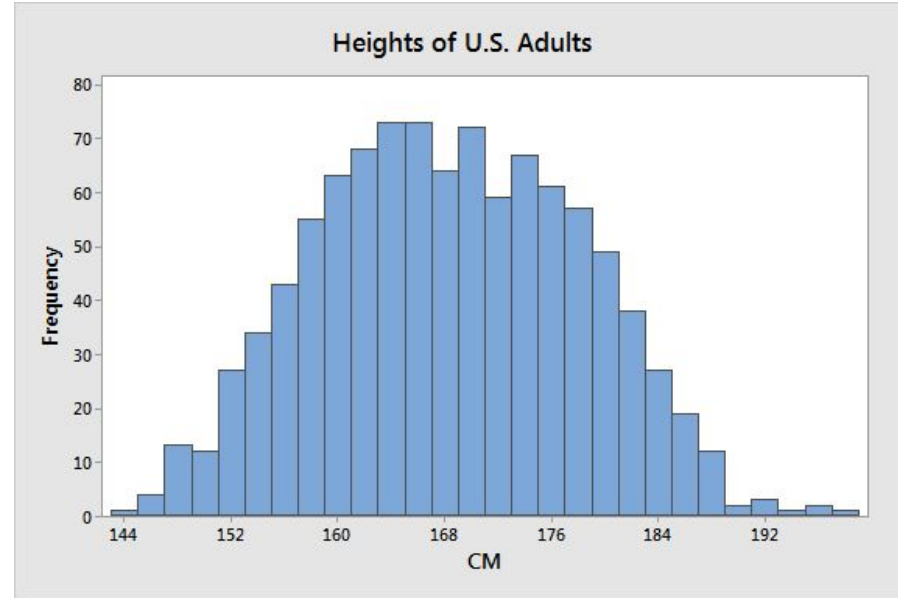
What is a population?

A scientist's job: collect finite data, and make general statements about the world

Consider someone studying the height of the population of the U.S.

This is a finite population with a well-defined average.

But we can't collect all that data, and it is not meaningful.



<https://statisticsbyjim.com/basics/populations-parameters-samples-inferential-statistics/>

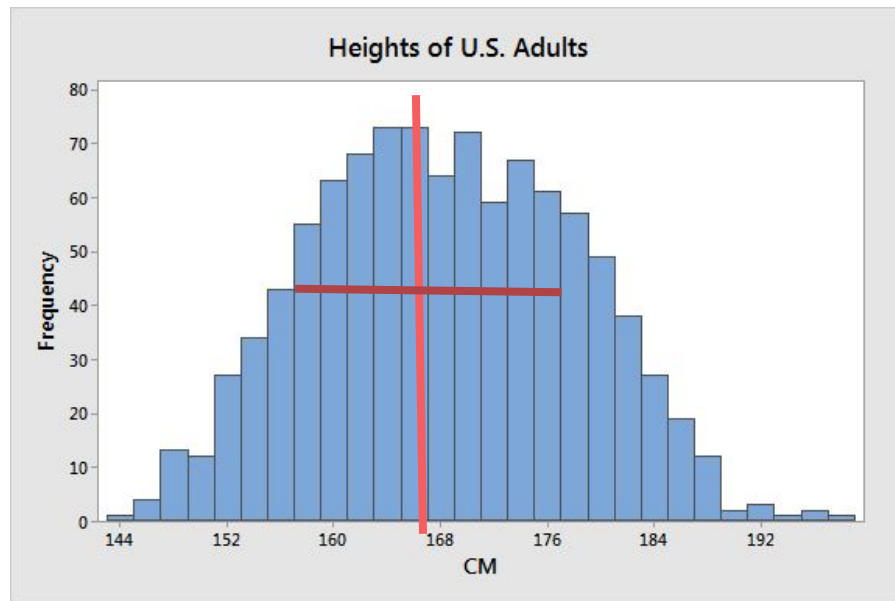
Finite population and frequency distributions

Sample average (mean)

Sample variance

$$\bar{x} = \frac{1}{n} \left(\sum_{i=1}^n x_i \right) = \frac{x_1 + x_2 + \cdots + x_n}{n} = \mu$$

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2$$



Infinite population and density distributions

Imagine we have infinite people

Imagine the histogram bins to become infinitely thin, but somehow not infinitely short

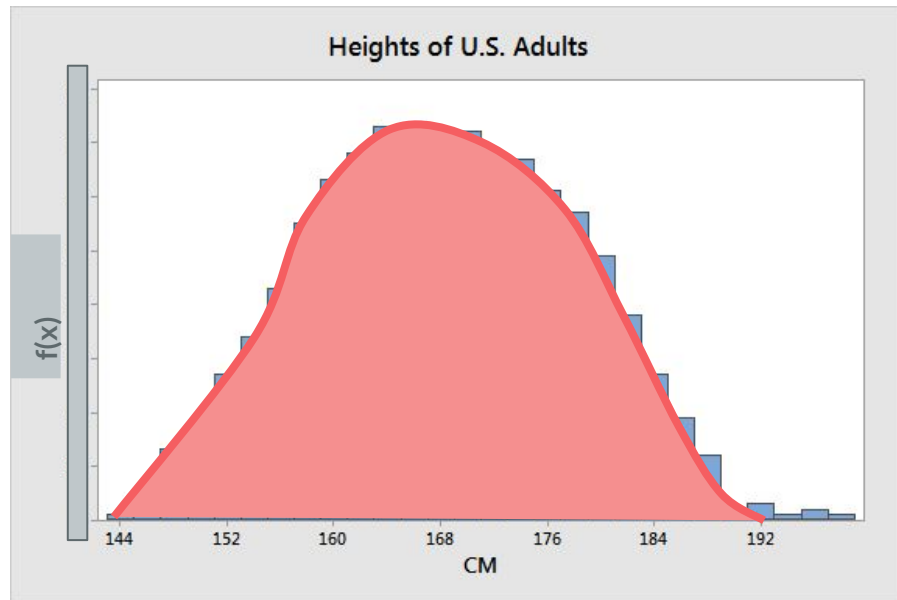
“Sampling with replacement”

Make an analytical function instead of histogram

-> Frequency density distribution

$$\mu \equiv E[X] = \int_{-\infty}^{+\infty} x f(x) dx$$

$$\sigma \equiv \sqrt{E[(X - \mu)^2]} = \sqrt{\int_{-\infty}^{+\infty} (x - \mu)^2 f(x) dx},$$



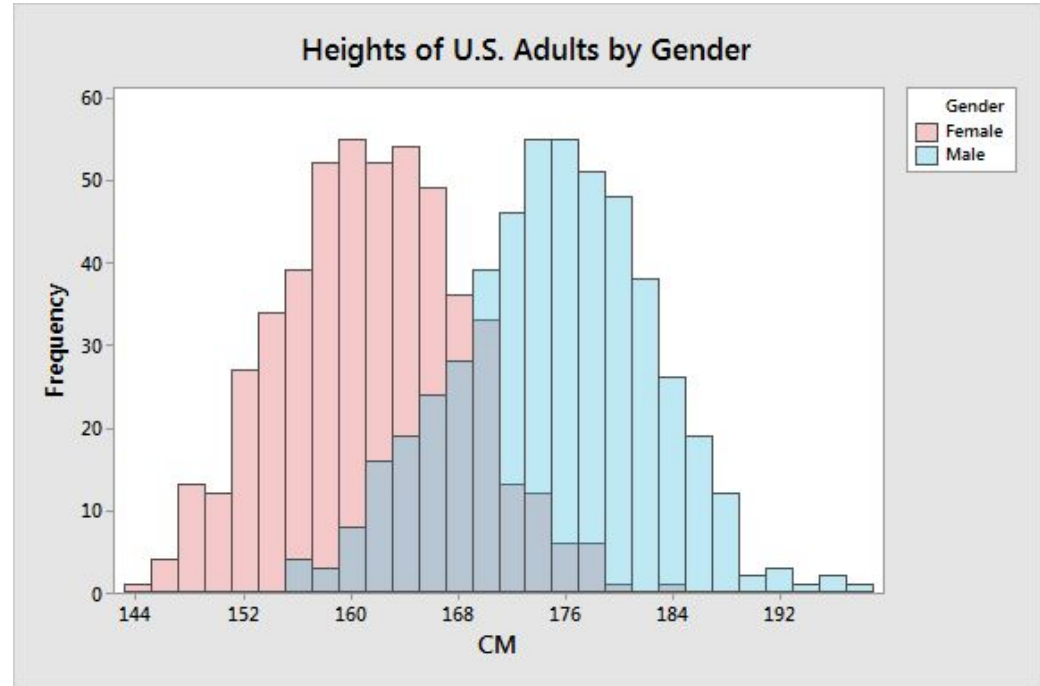
Sampling “effects”

Data can be biased compared to what we think it is

Data can include sub populations

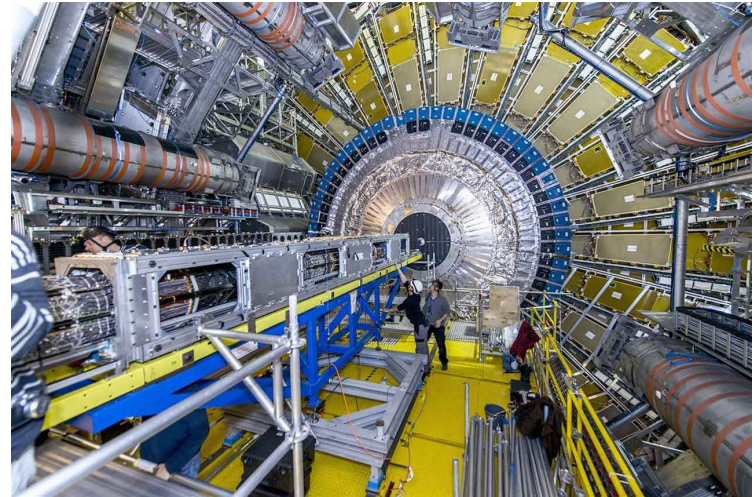
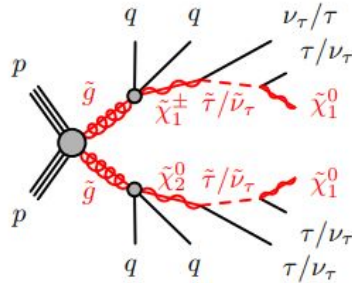
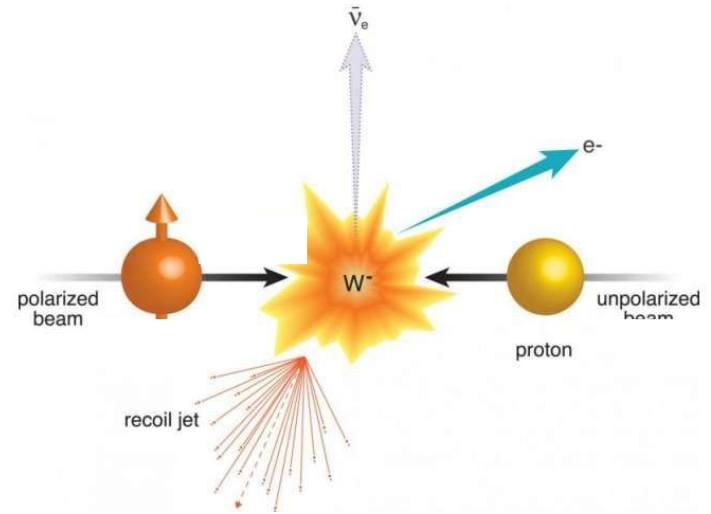
Finite data is **never** a perfect representation of the supposed frequency density distribution

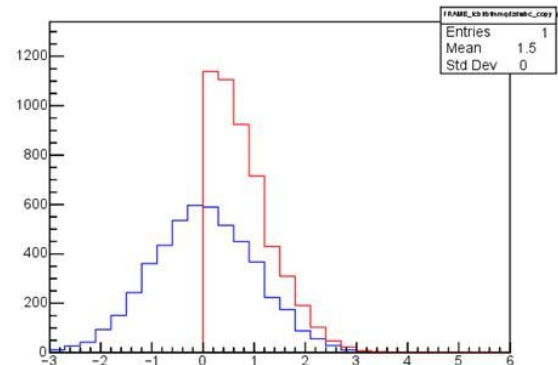
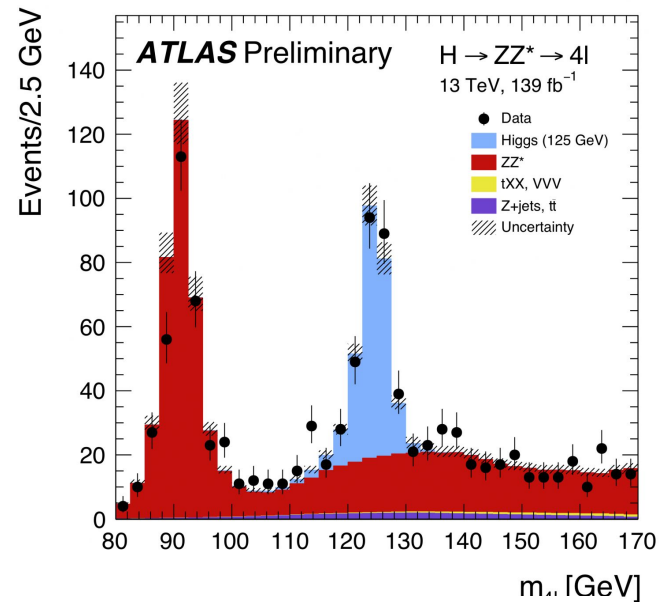
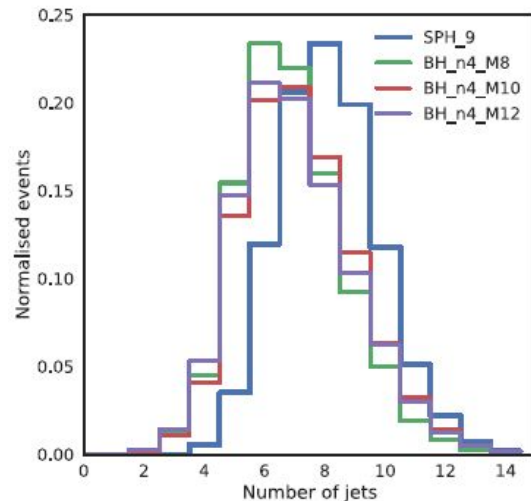
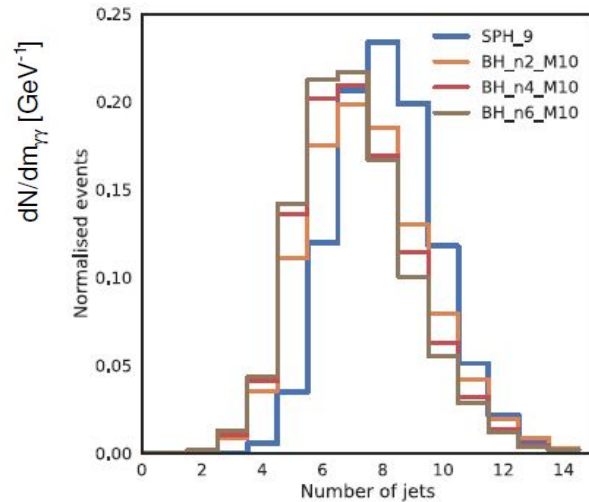
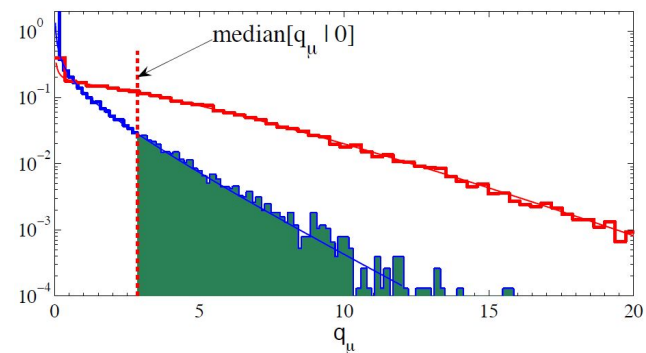
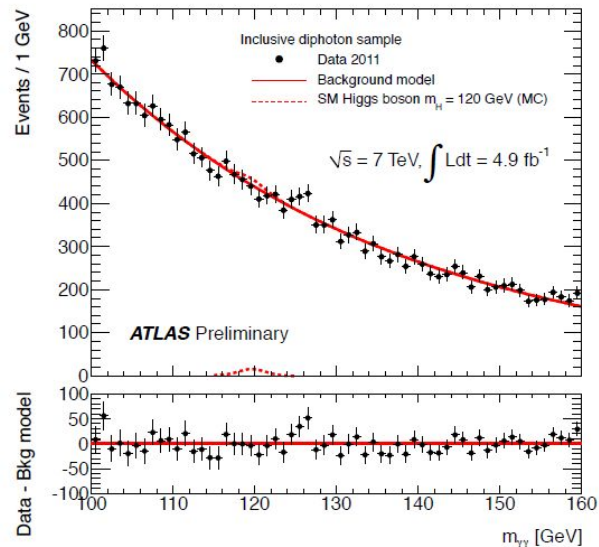
How can we then find a good function to represent our population?



ATLAS experiment

1. Accelerate protons in two directions to almost the speed of light
2. Smash them together inside a detector
3. Collect data from the debris
4. Analyze the data to figure out what happened





Event classification

Try to separate the different kind of events that were produced

This is really difficult because of the complexity of the data!

What are the right “cuts” to make to get as clean a distribution as possible?

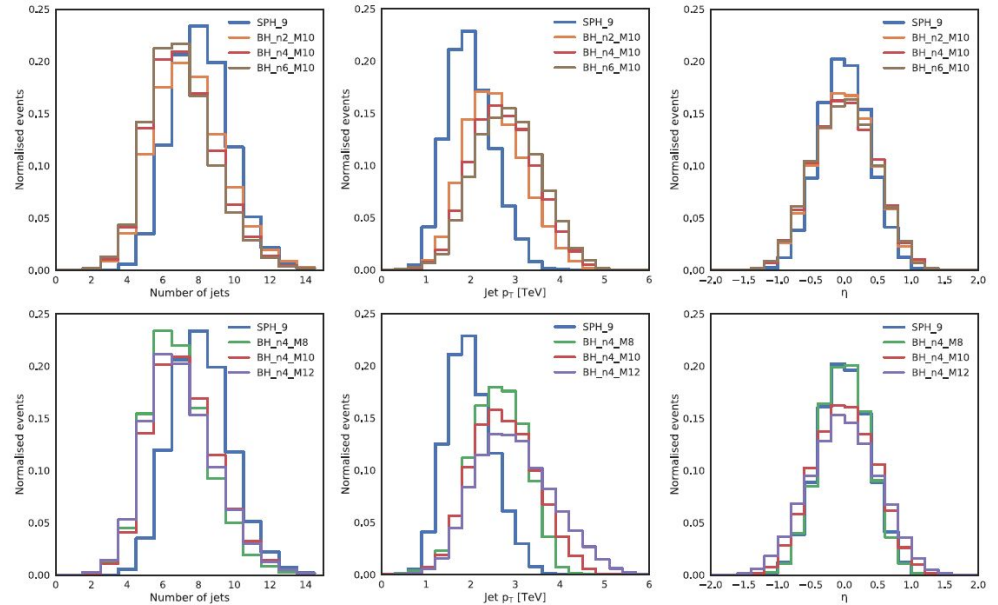


Fig. 1 Normalised distributions of the number of signal jets (left), the p_T of all signal jets (middle) and the pseudorapidity η of the highest- p_T jets (right). The sphaleron events are shown with blue histograms. The

plots in the upper panel show the BH scenarios with $n = 2$ (orange), 4 (red) and 6 (brown), while those in the lower panel show the BH scenarios with $M_{\min} = 8$ (green), 10 (red) and 12 (purple) TeV

Machine learning classification of sphalerons and black holes at the LHC

Aurora Singstad Grefsrud¹, Trygve Buanes¹, Fotis Koutroulis³, Anna Lipniacka², Rafał Masełek^{3,5}, Andreas Papaefstathiou⁴, Kazuki Sakurai^{3,a}, Therese B. Sjursen¹, Igor Slazyk¹

Machine learning

Deep learning model was quite good at this, but it just says which population an event belongs to...

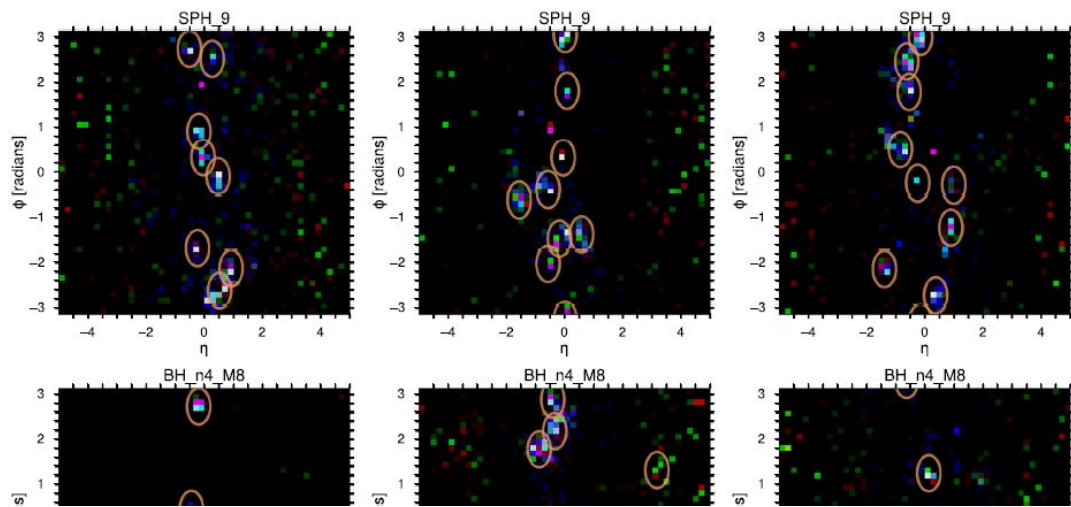
What did it learn?

Something interesting?

Would it work for new events?

How certain are the predictions?

I should instead model populations!



Eur. Phys. J. C (2024) 84:442

Page 9 of 15 442

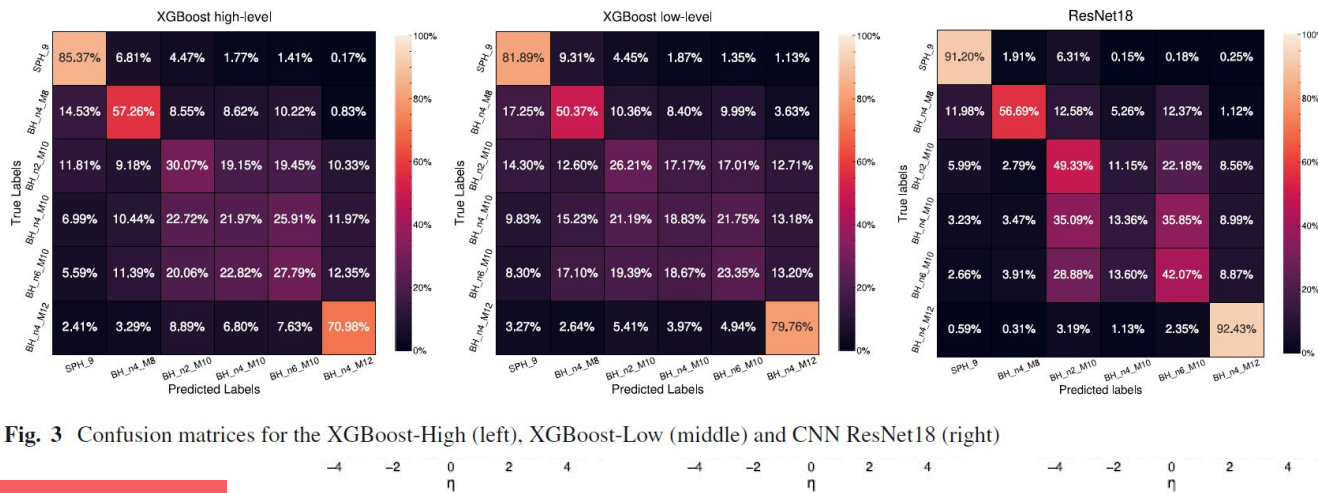


Fig. 3 Confusion matrices for the XGBoost-High (left), XGBoost-Low (middle) and CNN ResNet18 (right)

Does “infinite population” approximation make sense for us?

- SO much data! 1000 datapoints/second recorded by the detector
- There are basically infinite protons to collide
- Each collision is independent (unbiased)

However

- Interesting data is in the “tail” of the energy distribution (most collisions are not hard collisions)
- Interesting data is very unlikely even in hard collisions
- We have to filter the data, effectively creating “contaminated” data sets with several populations because we don’t have access to “labels”

Nonparametric statistics

Confusing term...

1. A method of statistical modeling where we do not assume any specific **shape** to the function describing the distribution the data is drawn from. “Distribution free”.
2. A method of statistical modeling where we do not assume any specific **structure** to the model. Individual parts of the model are typically assumed to have a specific shape. The complexity of the model grows with the complexity data. Includes non-parametric hierarchical Bayesian models.

In this work, we use definition nr 2. In ATLAS experiment we use definition 1.

Bayesian inference

Very satisfying theory-wise

You can calculate anything!

Very scary computational-wise

You must take into account everything...

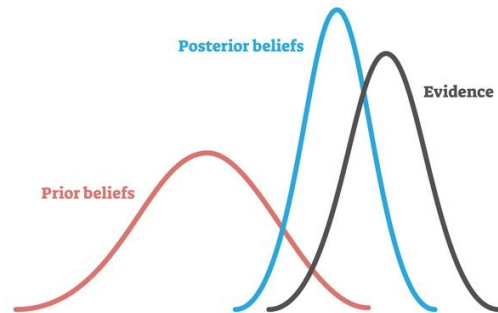
- **Proposition:** a logical True/False statement such as "It is raining"
- **Plausibility:** the degree of belief we have in that proposition being true. This degree of belief may vary from complete certainty in the statement being False, through complete uncertainty to complete certainty of the statement being True.
- **Probability:** the mathematical representation of plausibility as a monotonically increasing function from 0 to 1, with 0 being complete certainty that the proposition is False, 0.5 complete uncertainty and 1 complete certainty in it being True. Probabilities are calculated using the sum and multiplication rule.
- **Inference:** the calculations of probabilities related to a problem. The point of inference is to quantify our degree of belief in each proposition so we may make informed choices. Inference is not about finding the correct proposition. Problems that have one correct solution, which may be calculated fully, fall under the category of deductive problems. I believe most non-trivial problems are problems of inference. In any case, most problems scientists are interested in are problems of inference.

If we knew the probability of getting any x (of unspecified dimensionality for now) sampled from a population f , $P(x|f, D, I)$, as well as the probability of f , $P(f|D, I)$, both conditional on the data we have, we could calculate the probability regardless of the population:

$$P(x|D, I) = \sum_f P(x|f, D, I) \cdot P(f|D, I) \quad (4)$$

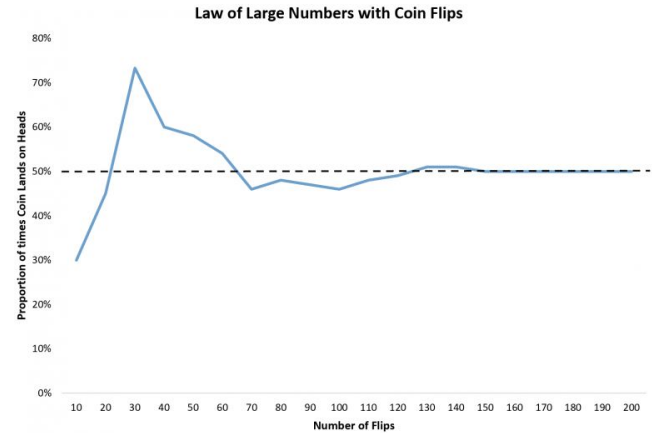
Some assumptions must be made about the possible populations f if we are to even attempt to start this calculation. These will all be contained in the conditional I , and we should remember that all results are dependent on these assumptions.

BAYESIAN ANALYSIS

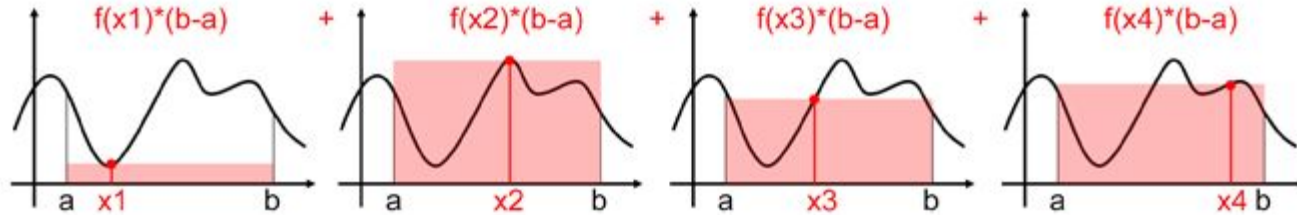


Monte Carlo sampling

Monte Carlo sampling lets us estimate calculations that would take a long time or be impossible to do analytically.



<https://www.statology.org/law-of-large-numbers/>



$$\frac{1}{4} * (\text{rectangle 1} + \text{rectangle 2} + \text{rectangle 3} + \text{rectangle 4}) \approx \text{area under curve}$$

© www.scratchapixel.com

Sample randomly, then take the average. The method should converge to the true value in the limit of infinite samples.

<https://www.scratchapixel.com/lessons/mathematics-physics-for-computer-graphics/monte-carlo-methods-in-practice/monte-carlo-integration.html?url=mathematics-physics-for-computer-graphics/monte-carlo-methods-in-practice/monte-carlo-integration>

Markov Chain Monte Carlo

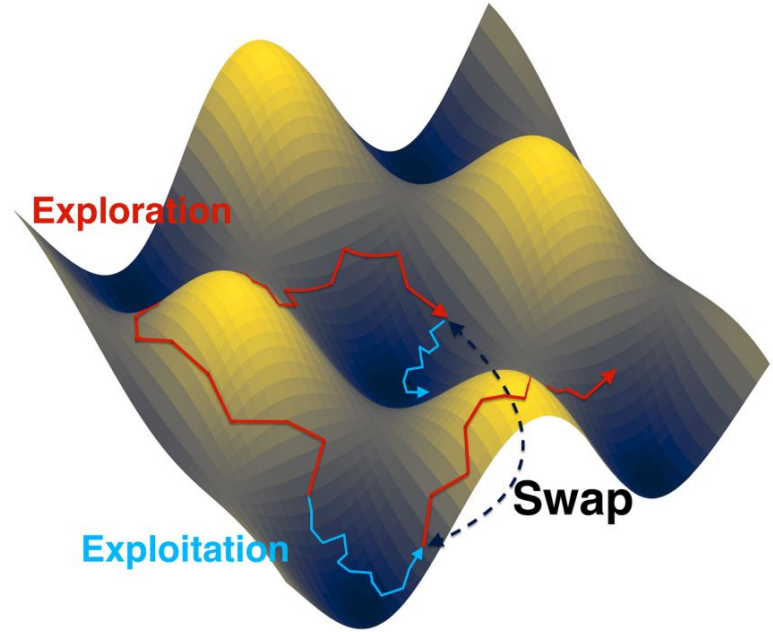
- Make a step in a random direction
- Depending on your new position, get new rules for random stepping
- Make a new step
- Repeat

Each step is only dependent on the previous step.

Example: protein folding

Different from gradient descent, which always goes in the “optimal direction”

Much faster than naive Monte Carlo

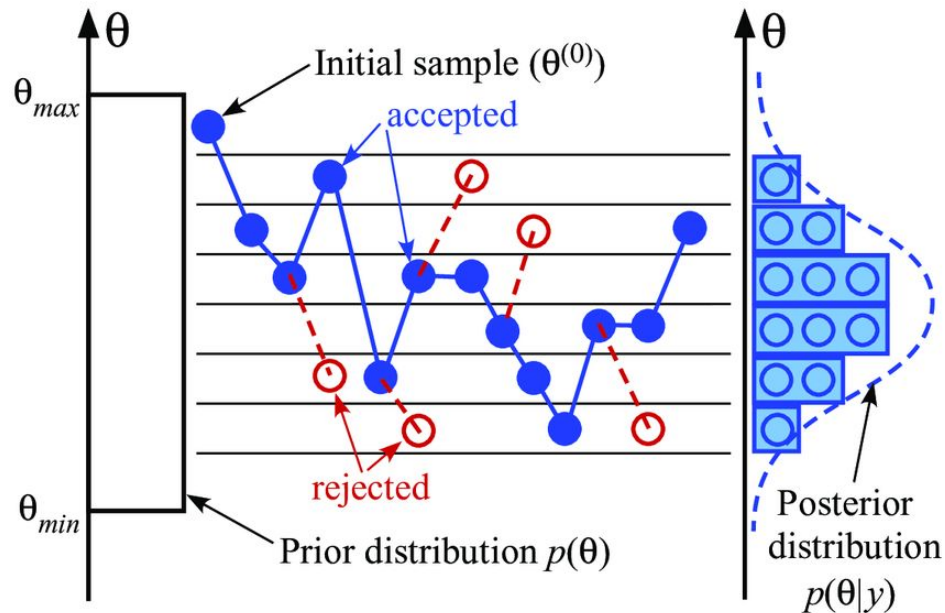


<https://arxiv.org/abs/2008.05367v3>

Gibbs sampling

- Start by randomly sampling from a start (prior) distribution
- Save the sample
- Update the sampling distribution based on how well the current sample fits the data
- Make new sample from new distribution
- Keep going for a set amount of samples

The final distribution is the average of all the samples. Not all samples are kept.

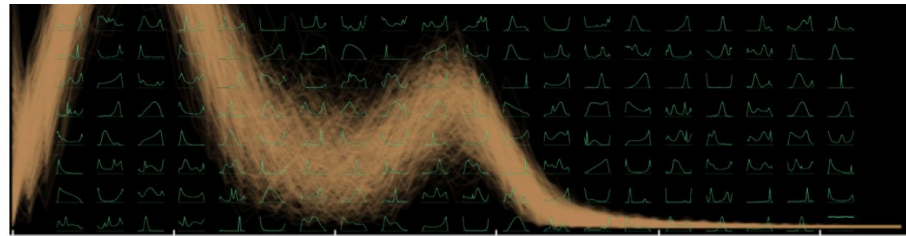
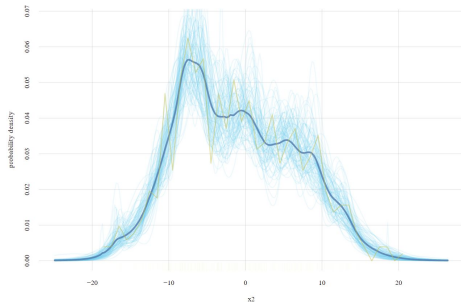


Bayesian Nonparametric modeling with Inferno

Automatically sets up the structure for the statistical model of population distribution estimation

Handles the MCMC simulation automatically

Produces plots like these:



Bayesian nonparametric inference

Documentation

This repository provides an R package and some theoretical background for exchangeability, or "inference about populations". The package is under a stable phase. This means that function names and arguments may still change.

<https://github.com/pglp/inferno>

I'm no longer involved in the development of this software



imgflip.com

@RPE-CLARK-TUMBLR

Math

1.
$$P(x_1, x_2, \dots, x_n | I) = \int \prod_{i=1}^n F(x_i) dQ(F | I)$$

2.
$$\int F \cdot p(F) dF = E(F)$$

3.
$$F(v_1, v_2, \dots, v_K) = \sum_{i=1}^l w_i \prod_{j=1}^K A_j(v_j, \alpha_{i,j})$$

4.
$$P(x | D, I) = \int F(x) \cdot P(F | D, I) dF = E(F(x)) \approx \langle F(x) \rangle = \sum_{i=1}^N \frac{F_i(x)}{N}$$

K and I are large, and grow with the number of input features. For simple data, 100s to 1000s of parameters.

Parameters



Inferno MCMC

- Gibbs sampling
- Specify desired number of samples
- Specify number of MCM chains
- More chains <- more random <- better approximation
- Basically ensembling of simulations

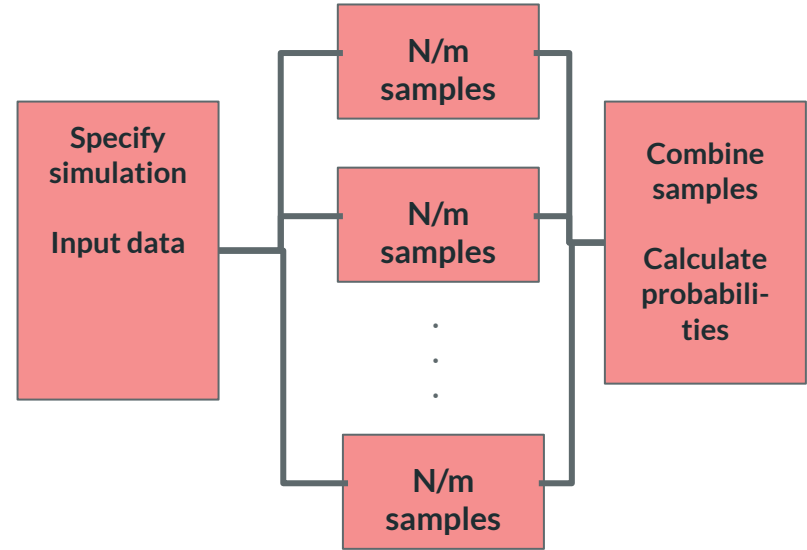
$$P(x|D, I) = \int F(x) \cdot P(F|D, I) dF = E(F(x)) \approx \langle F(x) \rangle = \sum_{i=1}^N \frac{F_i(x)}{N}$$

Number of
samples



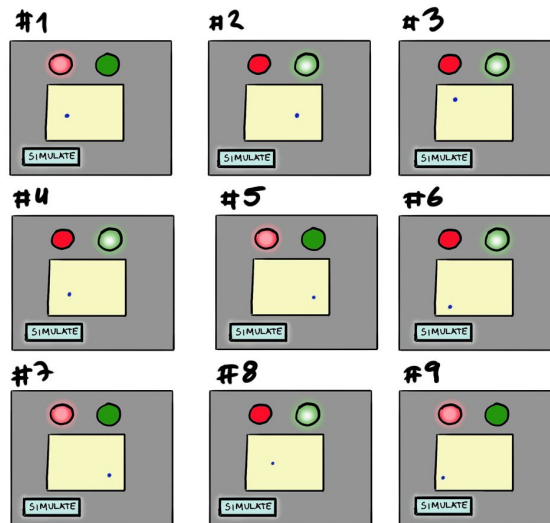
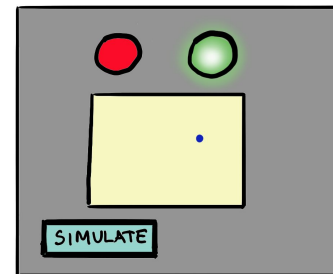
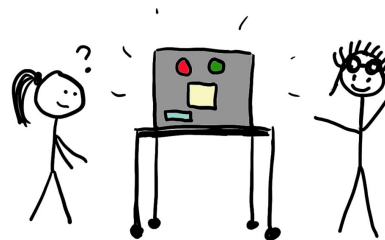
Computational efficiency and parallelisation

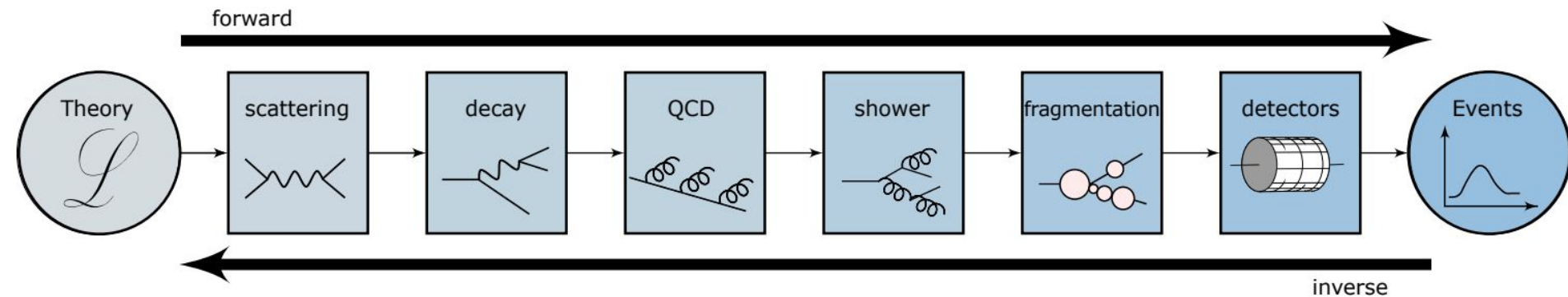
- Less samples <- faster (but worse)
- Divide number of samples by number of chains
- One CPU per chain
- Many CPUs <- many chains <- faster calculation
- If we do not split into chains, we can not utilize parallelisation



My study

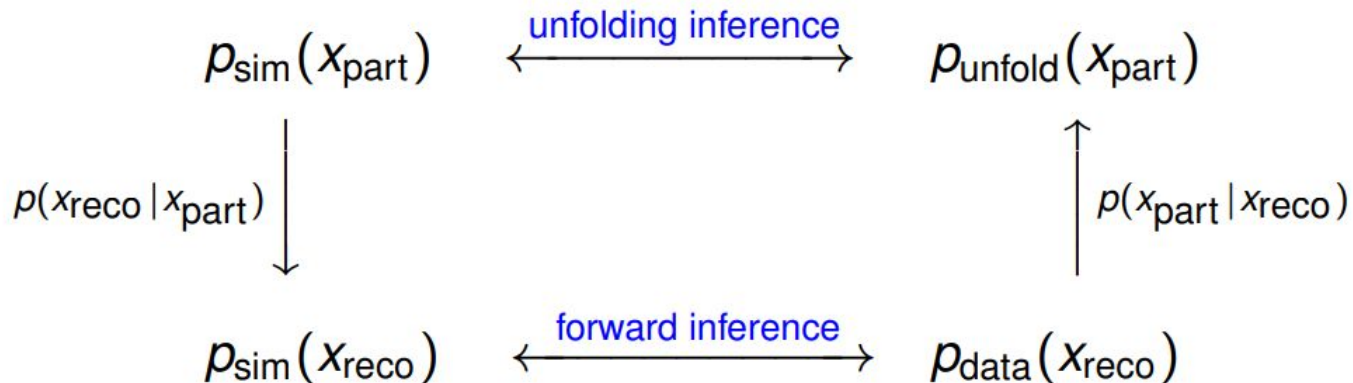
- What is the essence of event classification in High Energy Physics?
- To go from one probability distribution to another
- Very complex data! Hard to evaluate if methods even work.
- Toy distribution
- Check asymptotic behaviour (limit in large number of data)
- Also check effects of bias, and “generalization ability”





This is related, I promise!

- four phase space distributions



The data

- “Robot arm”
- Inverse problem
- With noise
- Try to separate the two populations!
- This is an ongoing study and part of my PhD

Distributed features:

$$C \in \{ \text{"red"} \text{ "green"} \}$$

$$\theta_1 \in [0, 2\pi) \subseteq \mathbb{R}^1$$

$$\theta_2 \in [0, \pi) \subseteq \mathbb{R}^1$$

$$\vec{x} = (x_1, x_2) \in X \subseteq \mathbb{R}^2$$

$$r_1 \in (2r_2, \infty) \subseteq \mathbb{R}^1$$

Known distributions:

$$P(c) = \begin{cases} 0.5, & c = \text{"red"} \\ 0.5, & c = \text{"green"} \end{cases}$$

$$p(\theta_1) = 1/2\pi$$

$$p(\theta_2) = 1/\pi$$

Changed

$$p(r_1 | c) = \frac{(r_1 - 2r_2)^{k_c - 1} e^{-r_1/c}}{s^{k_c} \Gamma(k_c)}$$

$$k_c = \begin{cases} 5, & c = \text{"red"} \\ 3, & c = \text{"green"} \end{cases}$$

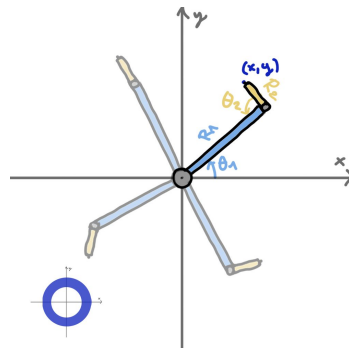
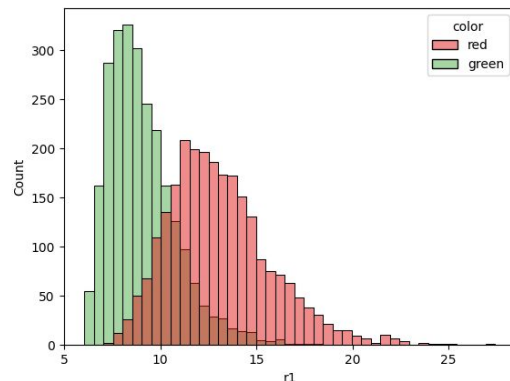
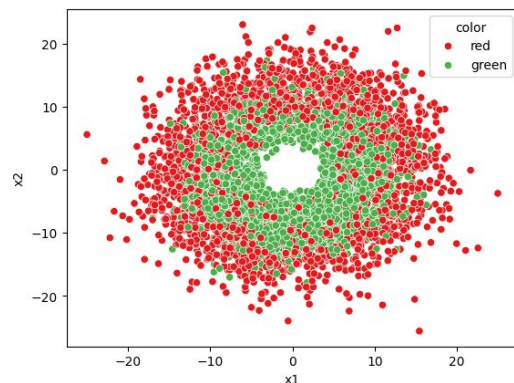
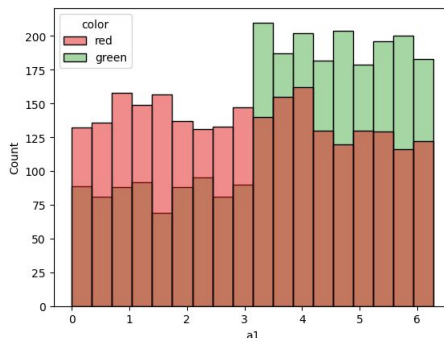
$s = \text{constant}$

Functional dependence:

$$\vec{x} = \vec{g}(\theta_1, \theta_2, r_1) = (g_1, g_2) = (x_1, x_2)$$

$$g_1 = r_1 \cos \theta_1 + r_2 \cos(\theta_1 + \theta_2)$$

$$g_2 = r_1 \sin \theta_1 + r_2 \sin(\theta_1 + \theta_2)$$



Can the machine learn?

What did it learn?

Did the computations take forever?

Was I able to figure out the TORQUE job description commands?



► TO BE CONTINUED

Experimental setup

Bayesian Nonparametric Inference software

R script

Computational setup

Shared file system

Containerization

Job scheduling

Data analysis

Results

Bonus results

Evaluation of computational solution

Time management and reduction of annoyance

Reproducibility

